

UNIVERSITY OF GHANA

(All Rights Reserved)

- **COURSE CODE AND TITLE:** CPEN 403 – Embedded Systems
- **COURSE INSTRUCTOR:** Dr. Godfrey Mills
- **LAB INSTRUCTOR:** Kelvin Prince Boateng

- **STUDENT:** Adika Nathaniel - 10957036

- **PROJECT:**INTRUDER DETECTION SYSTEM USING LDR SENSORS

1.INTRODUCTION

In today's world, security has become an essential aspect of both residential and commercial environments. Unauthorized access, theft, and intrusions pose significant threats to property and personal safety. As a result, security systems have evolved to incorporate various sensors and technologies to detect and respond to potential security breaches. One such approach involves monitoring changes in light intensity to identify suspicious activities, such as the use of flashlights by intruders or unexpected light fluctuations in protected areas.

The **Light Intensity Security Detection System** is an embedded system designed to monitor ambient light levels and trigger an alarm when a significant increase in light intensity is detected. The system employs a **Light Dependent Resistor (LDR) sensor**, which is highly sensitive to variations in light. Under normal conditions, the system continuously tracks the surrounding light levels. However, when an abrupt increase in illumination is detected—such as when an intruder uses a flashlight or when unauthorized access leads to unexpected lighting—the system enters **alarm mode**, triggering both auditory and visual alerts.

The system is built around an **Arduino Uno microcontroller**, which processes real-time data from the LDR sensor and determines whether the detected light intensity exceeds a predefined threshold. Once the threshold is surpassed, the Arduino activates a **buzzer**, which emits an audible alarm with a modulated frequency, and a **red LED**, which blinks to indicate an alert state. Additionally, an **LCD screen** is integrated into the system to display real-time status updates, including normal operation, detection alerts, and system resets.

For enhanced functionality, the system incorporates a **button interface** that allows users to check the total number of detections recorded. This feature provides valuable insight into the frequency of detected security events over time, assisting users in analyzing intrusion patterns.

To improve reliability and energy efficiency, the system utilizes **Watchdog Timer and Low-Power Modes**. The **Watchdog Timer (WDT)** ensures that the microcontroller remains operational by automatically resetting it in case of software malfunctions or unexpected freezes, thereby enhancing system stability. Meanwhile, the **Low-Power Mode** optimizes energy consumption, making the system more suitable for long-term deployment, especially in battery-powered applications.

The **Light Intensity Security Detection System** is designed to be simple yet effective, offering a cost-efficient solution for security applications. It can be deployed in a variety of environments, including residential buildings, offices, storage facilities, and restricted areas. Its ability to detect unauthorized illumination makes it particularly useful in scenarios where traditional motion or infrared-based sensors may not be ideal.

Overall, this system serves as an innovative approach to intrusion detection by leveraging light intensity monitoring. By integrating real-time data processing, auditory and visual alerts, and low-power functionalities, the system provides a reliable and responsive security solution to help safeguard protected areas.

2.PROJECT DEFINITION AND SYSTEM FUNCTIONALITY

The **Security Detection System Using an LDR and Arduino** is a light-sensitive intrusion detection system designed to enhance security in various environments. It operates by continuously monitoring ambient light levels and detecting sudden, unauthorized fluctuations, such as those caused by an intruder using a flashlight. The system processes these signals, applies filtering techniques, and triggers alarms when necessary, ensuring an efficient and responsive security solution.

SYSTEM FUNCTIONALITY:

1. **Light Monitoring:**
   The system employs a **Light Dependent Resistor (LDR)** to continuously measure the intensity of surrounding light. The LDR, a passive sensor, detects changes in illumination and converts them into electrical signals, which are then processed by the **Arduino Uno microcontroller**. This feature enables real-time tracking of environmental light conditions and immediate identification of anomalies.
2. **Signal Processing:**
   To ensure accurate detection, the LDR signal undergoes **signal conditioning and filtering**. The system integrates a **biquad filter** to eliminate unwanted noise and isolate specific frequency components. Notably, a **3 Hz signal**—often associated with intruder movement and flashlight flickering—is extracted and analyzed. This approach enhances the system's ability to differentiate between normal and suspicious light variations.
3. **Alarm Triggering:**
   The system utilizes **hysteresis logic** to determine whether a detected change in light intensity is significant enough to trigger an alarm. When a predefined threshold is exceeded, the system enters **alarm mode**, activating:
   - A **buzzer** that produces a modulated tone to provide an audible alert.
   - A **red LED** that blinks or fades to serve as a visual indicator of the detected intrusion.
     These alarm mechanisms ensure that security personnel or property owners are promptly alerted to potential threats.
4. **Status Indication:**
   To provide clear system feedback, the **status of the security system** is indicated using both LEDs and an **LCD display (16×2)**.
   - A **green LED** remains active under normal operating conditions, signifying that no threats are detected.
   - The LCD screen displays real-time messages, such as:
     - **"Light Intensity: [Value]"** – showing the current light level.
     - **"Status: Normal"** – indicating stable conditions.
     - **"ALARM: DETECTED"** – warning users of a security breach.
5. **Data Logging and Retrieval:**
   To enhance security monitoring, the system includes a **data logging feature** that records detection events. When an intrusion is detected, relevant data, including the **time of detection and the number of occurrences**, is stored in the **EEPROM** (Electrically Erasable Programmable Read-Only Memory). A **push-button interface** allows users to retrieve and display this stored data on the **LCD screen**, providing

In essence, the **Security Detection System Using an LDR and Arduino** is an efficient, real-time intrusion detection solution that leverages light intensity monitoring to identify unauthorized access. By integrating **signal processing techniques, intelligent alarm mechanisms, and data logging capabilities**, the system offers a robust and reliable approach to security monitoring. Its ability to process light signals and differentiate between normal and suspicious activity makes it a valuable addition to residential, commercial, and industrial security systems.

3.OBJECTIVE OF THE DESIGN

- **Primary Objective:**
  To design a robust, reliable, and energy-efficient security detection system that:
  - Accurately detects changes in ambient light (indicative of an intruder) by filtering noise.
  - Provides immediate audible and visual alarms.
  - Logs detection events (count and time) for subsequent review.
- **Secondary Objectives:**
  - Ensure real-time processing via timer interrupts and efficient power management through sleep modes.
  - Use EEPROM to maintain data across power cycles.
  - Incorporate user interaction (via a button) for on-demand data retrieval.

COMPONENTS LIST

1. **Arduino Uno**
   - Microcontroller board (ATmega328P) used for reading sensor data, processing signals, driving outputs, and storing/retrieving data in EEPROM.
2. **LDR (Torch_LDR)**
   - A light-dependent resistor whose resistance varies with ambient light intensity.
3. **Resistors**
   - **R1**: Form part of the voltage divider or comparator input network for the LDR.
   - **R5** (or similarly named): Used with the green or red LED (current-limiting resistor).
   - Additional resistors as needed for pull-ups or setting thresholds.
4. **555 Timer (Optional in Some Designs)**
   - In the real world, the LDR (Light Dependent Resistor) might naturally experience a **3 Hz fluctuation** in light intensity—e.g., from movement or a flickering source. However, in Proteus, you can't always replicate all real-world ambient variations. Thus, the **555 timer** is used to **simulate** that **3 Hz signal** the LDR would see under actual conditions. By generating a stable 3 Hz wave, the 555 timer effectively stands in for the real-world frequency that would otherwise come from changing light patterns in an actual environment.
5. **LEDs**
   - **Red LED** (LED-RED): Used for alarm indication (blinking or fading).
   - **Green LED** (LED-GREEN): Indicates normal operation.
6. **Buzzer / Sounder (LS1)**
   - Produces an audible tone when driven by the Arduino (pin 6 in your schematic).
7. **16×2 LCD (LM016L)**

- Displays system status messages (e.g., “Light Intensity,” “ALARM: DETECTED,” or detection count/time).

11. **Button (BUTTON)**

- Normally open (NO) push-button connected to an Arduino digital input (pin 7 with internal pull-up).
- Pressing it triggers retrieval of detection count/time from EEPROM and displays on the LCD.

12. **Voltage Source (5 V Regulated Supply)**

- Powers the Arduino, comparator/op-amp, LCD, LEDs, and buzzer.

Additional Supporting Components

- **Decoupling Capacitors**: Small 0.1 μF capacitors near the power pins of the op-amp or the 555 timer for noise suppression.
- **Wires/Terminals**: For interconnections.

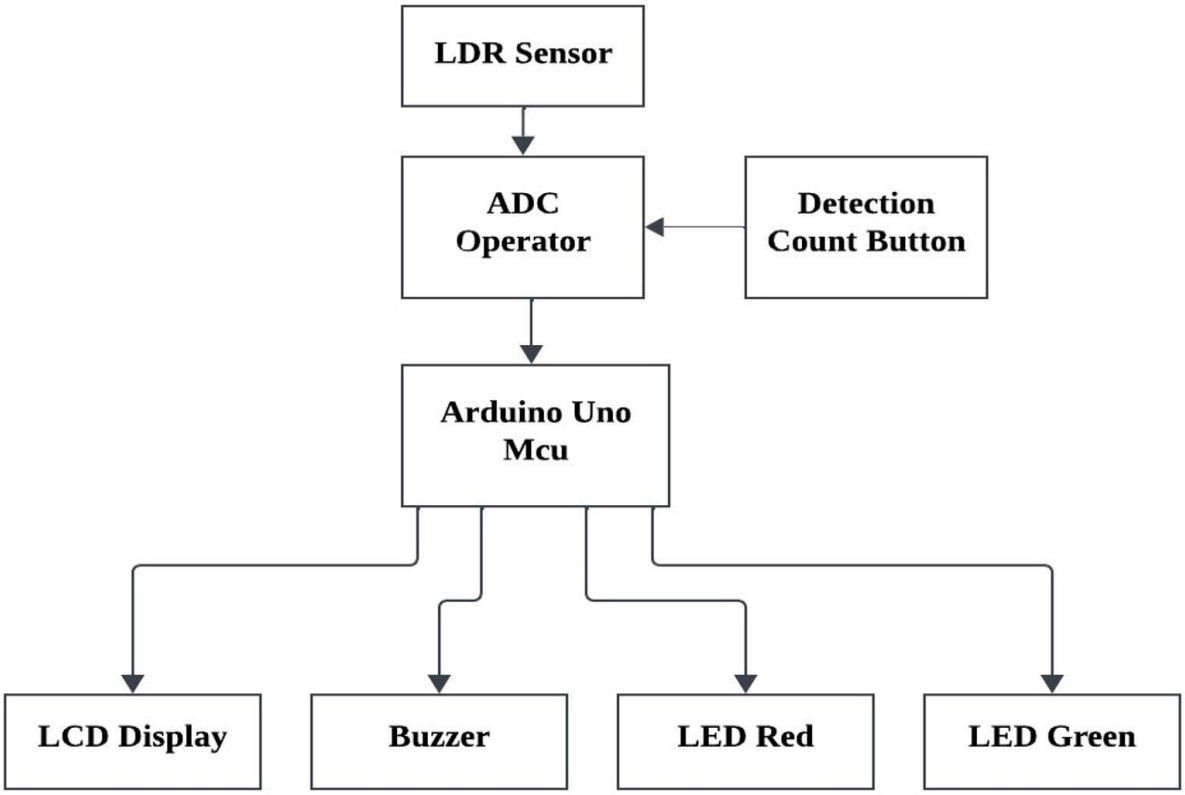
4.FUNCTIONS, INPUTS, AND OUTPUT REQUIREMENTS

- **System Functions:**
  - **Sensing:** Acquire the analog voltage from the LDR sensor circuit.
  - **Filtering and Detection:** Process the sensor data using a digital filter with hysteresis to detect a specific frequency signature (e.g., 3 Hz) associated with an intruder.
  - **Alarm Activation:** Trigger an alarm mode by:
    - Modulating a buzzer tone.
    - Controlling the red LED (blinking/fading) during alarm events.
  - **Status Indication:** Maintain a constant state in normal operation (e.g., green LED on, LCD showing “Status: Normal”).
  - **Data Logging:** Record detection events (count and detection time) in EEPROM.
  - **User Interface:**
    - Display status and event information on a 16×2 LCD.
    - Allow the user to press a button to retrieve and view the logged data.
- **Inputs:**
  - **LDR Sensor Signal:**
    - Analog voltage that varies with ambient light.
  - **Push-Button Input:**
    - A digital input (with internal pull-up) to request retrieval of detection log data from EEPROM.
- **Outputs:**
  - **Buzzer:**
    - Produces an audible tone with frequency modulation during alarm mode.
  - **Red LED:**
    - Provides a visual alarm indication (via PWM-controlled fading or blinking).
  - **Green LED:**
    - Indicates normal operation with constant brightness.
  - **LCD Display:**

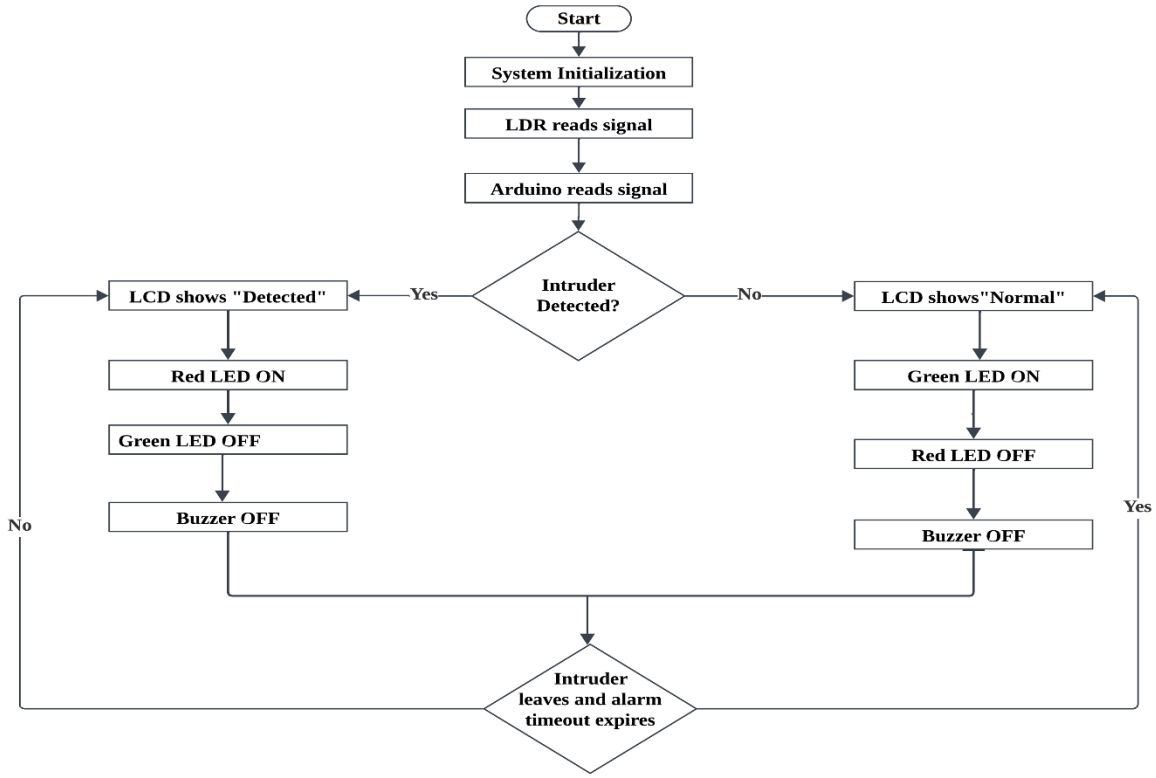
- Shows messages such as “Light Intensity”, “Status: Normal”, “ALARM: DETECTED”, and detection count/time when requested.

- **Serial Output (Optional):**
  - For debugging and monitoring system behavior during operation.
- **EEPROM Storage:**
  - Non-volatile storage of detection count and last detection time for retrieval even after power cycles

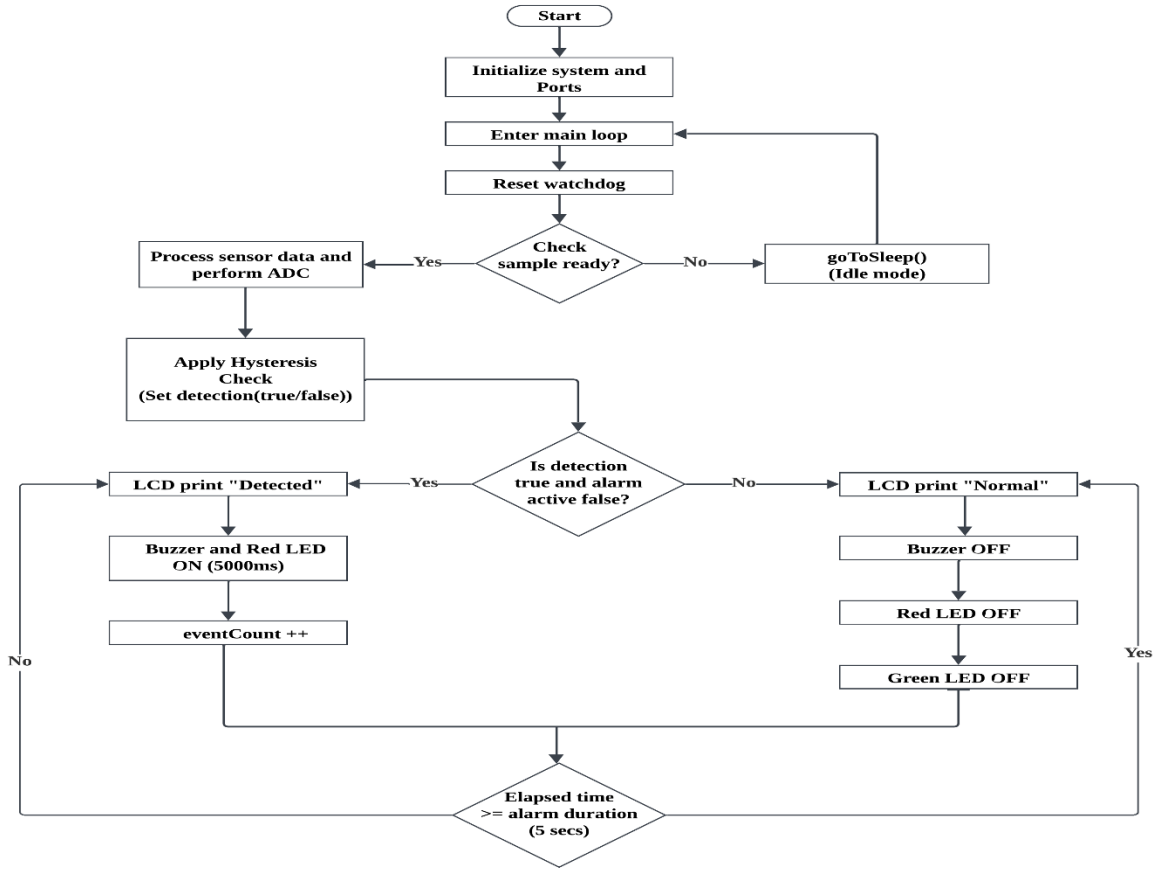
5.ARCHITECTURAL DIAGRAM



SYSTEM OPERATIONAL FLOW DIAGRAM

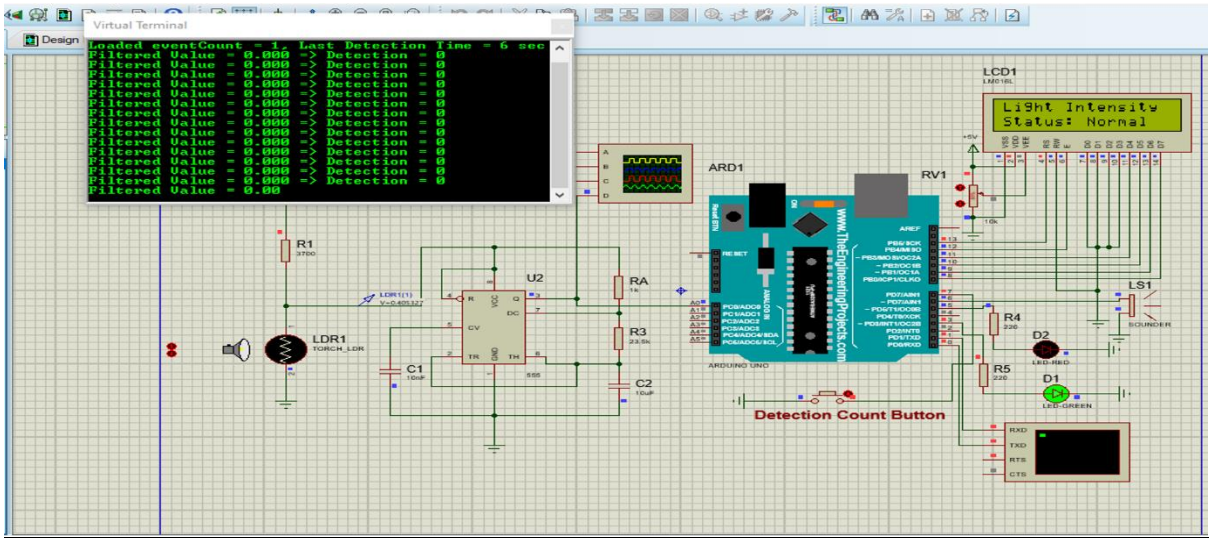


SOFTWARE FLOW DIAGRAM

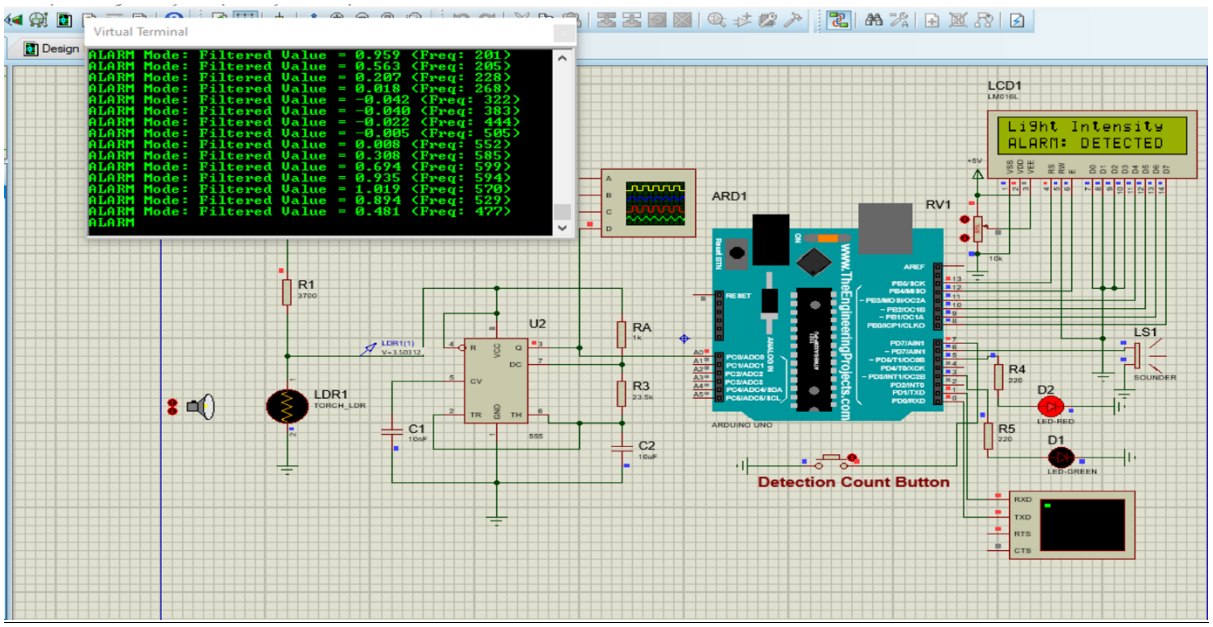


6.PROTEUS SIMULATIONS OF INTRUDER DETECTION SYSTEM

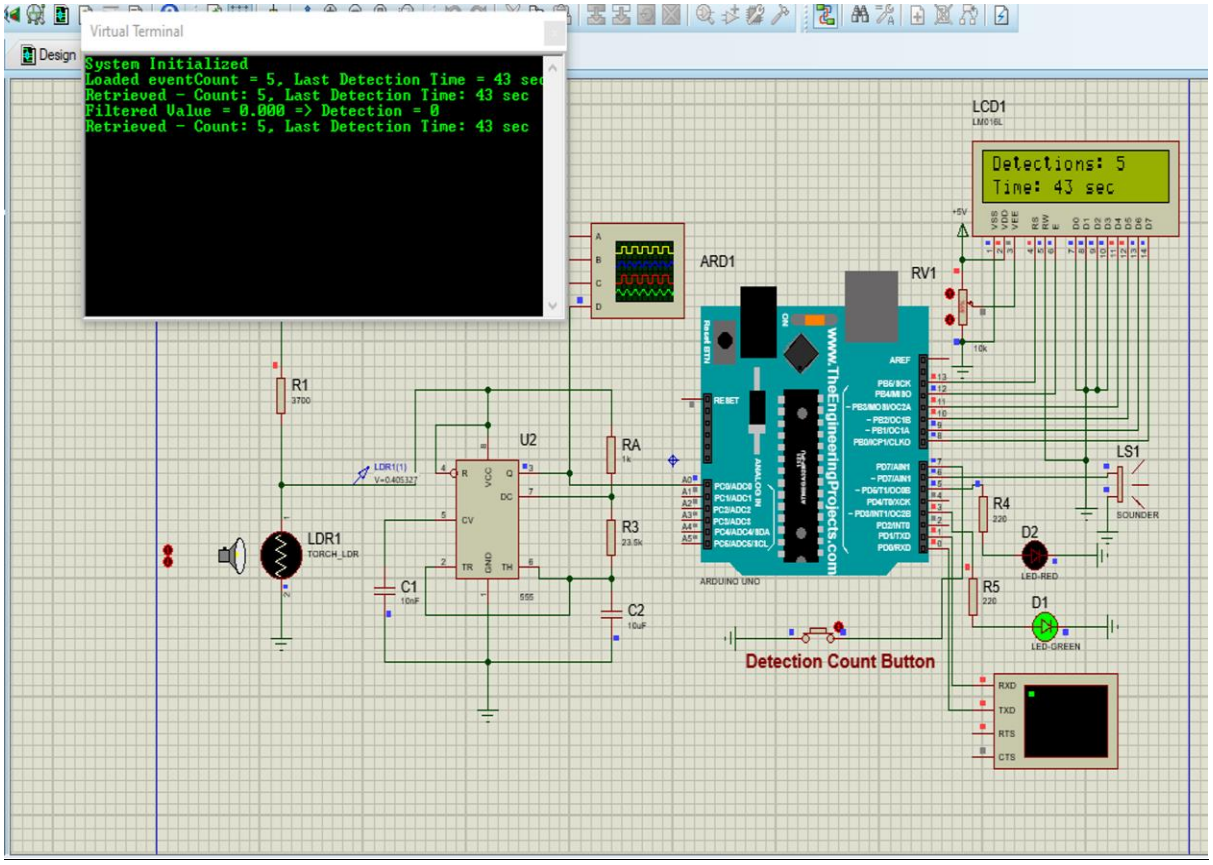
NORMAL LIGHT INTENSITY WITH NO INTRUDER INVADING THE AREA



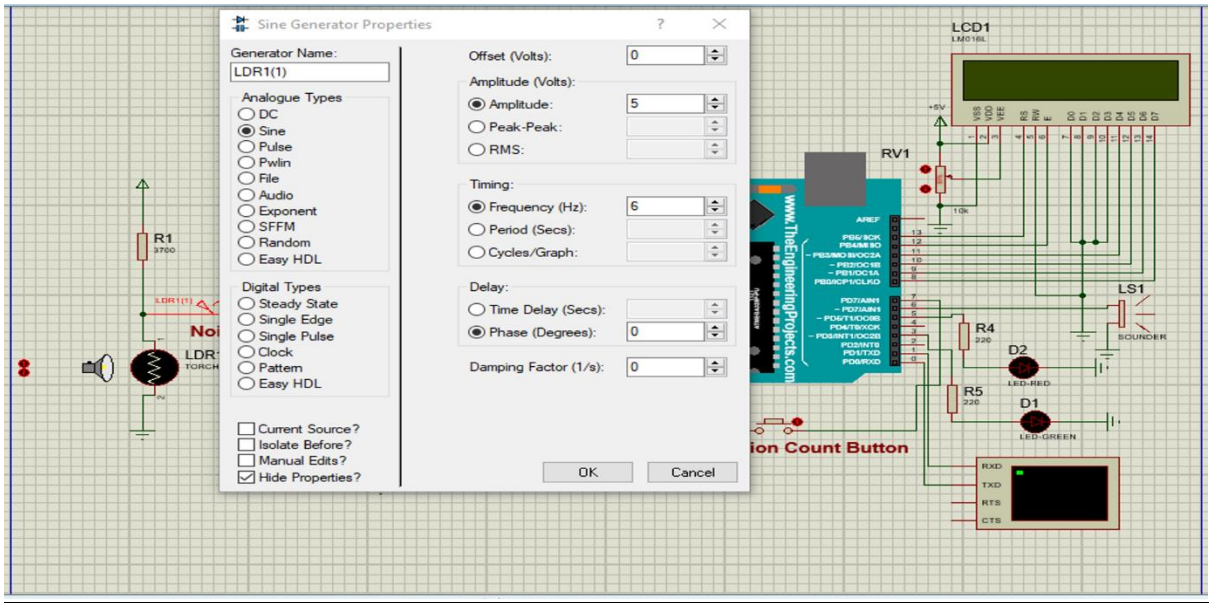
DARKNESS FALLING OR LOW INTENSITY BEING LOW INTRUDER PRESENCE



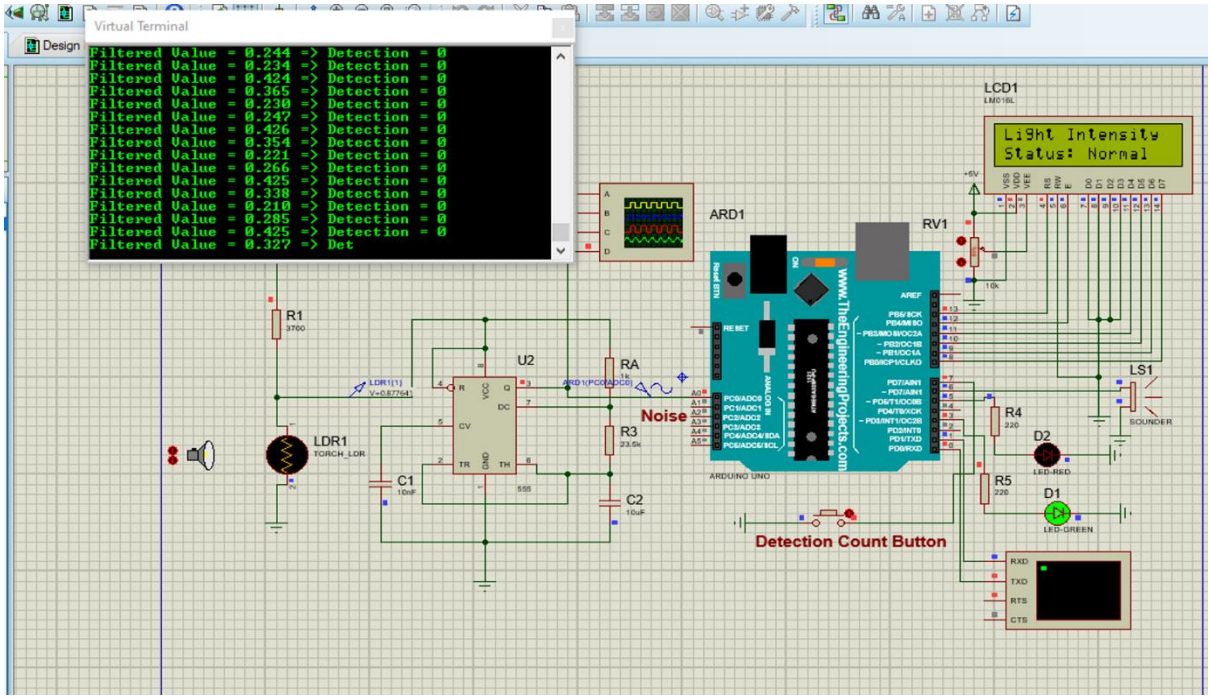
BUTTON PRESSED INDICATING NUMBER OF DETECTIONS ON THE LCD AND THE TIME THE DETECTION TOOK PLACE SINCE THE ARDUINO IS IN THE ACTIVE STATE



NOISE SIMULATION BY ADDING FREQUENCY OF 6Hz BEYOUND 3Hz CUTOFF FREQUENCY



FILTRATION OF THE 6Hz NOISE MAKING THE LDR APPEAR UNRESPONSIVE EVEN WHEN INTRUDER IS PRESENT



Explanation:

- **Real-World Behavior:** In an actual setup, the LDR circuit picks up a range of frequencies. The filtering circuit is designed to suppress unwanted noise (like a 6 Hz signal) while still passing the relevant intruder-induced frequency.

- **Simulation Limitation:** Proteus often simulates only one dominant frequency at a time. As a result, when the 6 Hz noise is applied, the filter removes it entirely, leaving little or no signal from the LDR.
- **System Impact:** Due to this, even though the system indicates an intruder, the LDR's output seems unresponsive in the simulation because the noise (and any mixed frequencies) isn't accurately represented.
- **Summary:** The filtering works as intended, but Proteus's limited frequency simulation makes it appear as if the system is unresponsive.

7.SYSTEM SPECIFICATIONS

1. **Power Supply**
   - **Voltage**: 5 V DC regulated supply (from Arduino or external source).
   - **Current**: Must be sufficient to power the Arduino Uno, the LCD (16×2), the buzzer, LEDs, and comparator/amplifier stages.
2. **Sensing & Detection**
   - **LDR (TORCH_LDR)**: Light-dependent resistor with a resistance range suitable for detecting significant light changes (e.g., 1 MΩ in darkness down to a few kΩ in bright light).
   - **Frequency Range**: Able to detect an intruder signal of around 3 Hz (or user-defined frequency) while filtering out noise (e.g., 6 Hz).
   - **Comparator/Op Amp (U2)**: Provides threshold-based or amplified signal to Arduino's ADC (depending on the exact design).
3. **Control & Processing**
   - **Arduino Uno** (ARD1):
     - 16 MHz clock, 2 kB SRAM, 32 kB flash.
     - 10-bit ADC for reading the LDR signal (0–1023 range).
     - Timer1 configured for a 100 Hz sampling rate.
     - EEPROM (1 kB) for event count and detection time storage.
     - Watchdog timer to reset system if main loop hangs.
4. **Outputs**
   - **Buzzer (Sounder)**: Capable of producing audible tones (400–600 Hz range) when modulated by Arduino.
   - **LED-RED**: Fades or blinks during alarm to visually indicate detection.
   - **LED-GREEN**: Indicates normal mode.
   - **LCD (LM016L)**: 16×2 display to show system status, detection events, and user-requested info (detection count/time).
5. **User Interaction**
   - **Detection Count Button**: Momentary push-button connected to Arduino digital pin with internal pull-up.
     - Pressing it triggers retrieval of detection count/time from EEPROM and displays on LCD.

8.FUNCTIONAL REQUIREMENTS

1. **Intrusion Detection**:

    1. System must detect a sudden change in light (above thresholdHigh) and trigger an alarm mode for 5 seconds.
    2. Filter out undesirable noise frequencies (like 6 Hz) so the system only responds to the specified intruder frequency (~3 Hz) or abrupt changes.
2. **Alarm Indication**:
    1. Buzzer must produce a varying tone during alarm.
    2. Red LED must blink or fade in alarm mode; green LED is off.
    3. LCD must display "ALARM: DETECTED."
3. **Normal Operation**:
    1. Buzzer off, red LED off, green LED on.
    2. LCD shows "Status: Normal" or "Light Intensity."
    3. **Data Logging**:
        - Must record detection count and last detection time (in milliseconds) in EEPROM.
        - Must persist across power cycles (non-volatile storage).
    4. **User Interaction**:
        - A button press retrieves detection data from EEPROM and displays it on the LCD (detection count + time).
4. **Performance Requirements**
    1. **Sampling Rate**: ~100 Hz to accurately read and filter the LDR signal.
    2. **Detection Response**: Must trigger an alarm within one or two sampling periods (≤ 20 ms) after crossing thresholdHigh.
    3. **Filtering**: Must sufficiently suppress noise signals like 6 Hz or spurious light flicker to avoid false alarms.
5. **Reliability & Safety Requirements**
    1. **Watchdog Timer**: Resets the system if main loop hangs, ensuring continuous operation.
    2. **Hysteresis**: Use thresholdHigh and thresholdLow to avoid rapid toggling.
    3. **Power Stability**: Must maintain correct operation at 5 V ± 5%.
6. **User Interface Requirements**

1**.LCD**: Must provide at least 16×2 characters to display status messages.

2**.Button**: Single press triggers retrieval of stored data and displays on LCD for ~3 s.

CONSTRAINTS

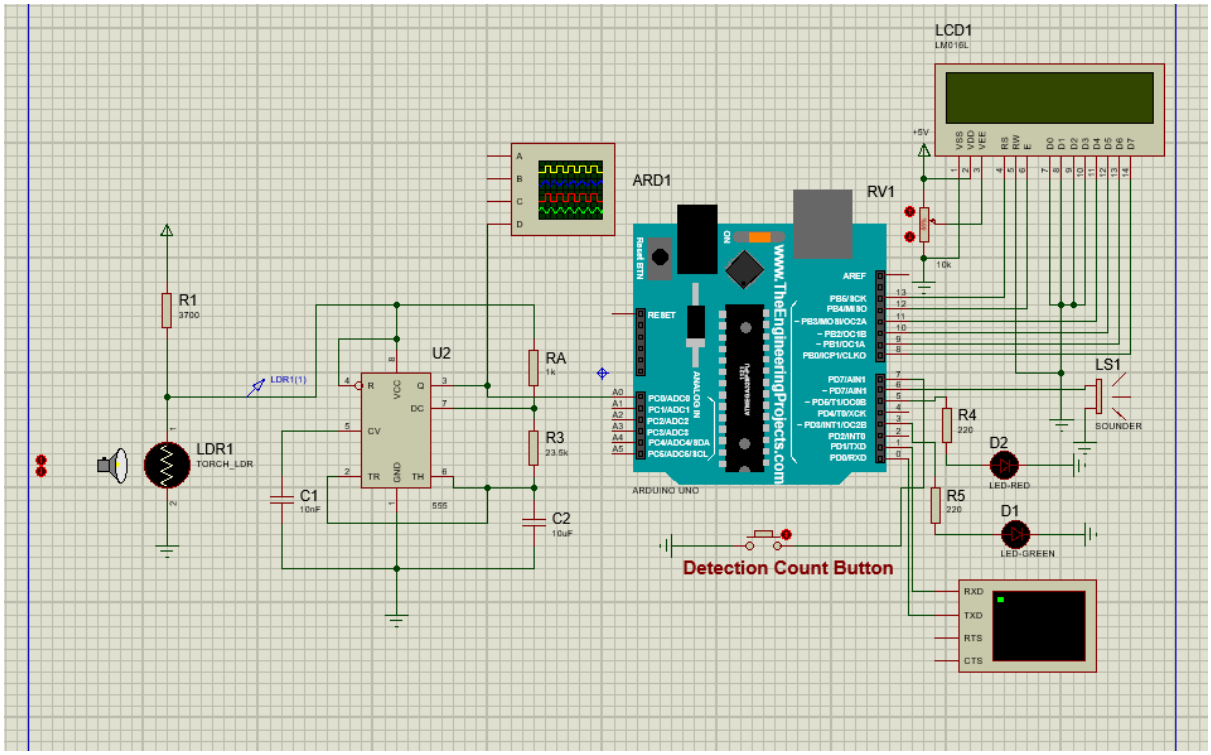
1. **Hardware Constraints**
    - **Arduino Uno Pin Availability**:
        - Buzzer on pin 6 (tone), red LED on pin 5 (PWM), green LED on pin 3 (PWM), button on pin 7, LCD on pins 13,12,11,10,9,8, etc.
    - **Limited EEPROM Size**:
        - Only 1 kB available. Must store minimal data (event count + last detection time).
    - **Op-Amp Comparator**:
        - In Proteus, real-time multi-frequency simulation is limited; must ensure noise frequencies are tested appropriately.
2. **Software Constraints**
    - **10-bit ADC** on Arduino for reading LDR (0–1023).

- **Prescaler** has a maximum value of 1024.
- **Clock frequency** is 16 MHz.
- **Timer1** used for sampling; cannot conflict with tone() or LED PWM timers.
- **Tone() Function** uses Timer2 internally, so LED pins using Timer2 (like pin 3) may conflict unless carefully assigned or swapped.

3. **Simulation Constraints**
   - **Proteus Frequency Limitations**:
     - Proteus does not perfectly simulate multiple frequency signals simultaneously, so certain test frequencies (like 6 Hz noise) might be completely filtered, making the system appear unresponsive under certain conditions.
   - **EEPROM Persistence**:
     - EEPROM contents persist between runs unless manually cleared or configured otherwise in Proteus.
4. **Timing & Power Constraints**
   - **Idle Sleep**: The system must enter low-power idle mode between Timer1 interrupts.
   - **5 s Alarm Duration**: Alarm must revert to normal mode automatically after 5 s.
   - **Watchdog Timeout**: 2 s, so main loop must execute `wdt_reset()` at intervals < 2 s.

9.SCHEMATIC DIAGRAM OF HARDWARE

Below is an **overview** of the main hardware blocks in the schematic:



1. **LDR Sensor Circuit**
2. **LDR1 (Light Dependent Resistor):** Changes its resistance based on light intensity.
3. **R2:** Form part of a voltage divider that feeds the Arduino's ADC.
4. **Arduino Uno (ARD1)**

5. **Microcontroller:** Reads the sensor signal on A0.
6. **Digital Outputs:**
    - **Buzzer on pin 6 (tone output):** Used in alarm mode with modulated frequency.
    - **Red LED on pin 3:** Blinks during alarm mode (if not reassigned).
    - **Green LED on pin 5:** Illuminated during normal operation.
    - **LCD on pins 13, 12, 11, 10, 9, 8:** Displays system messages.
    - **Digital Input for Push-Button:**
    - A push-button is connected to digital **pin 7** (using the internal pull-up resistor).
    - When pressed, the Arduino retrieves and displays the detection count and the last detection time (stored in EEPROM) on the LCD.
7. **Firmware:** Runs the embedded C++ code (sketch) that implements sensor filtering, hysteresis detection, alarm logic, and EEPROM logging of detection events.
8. **LCD (16×2)**
9. Displays "Light Intensity" on the first row.
10. Shows system status (e.g., "Status: Normal", "ALARM: DETECTED", or the detection count and time when the button is pressed) on the second row.
11. **Buzzer and LEDs**
12. **Buzzer (pin 6):** Used in alarm mode; its tone is modulated to produce a varying sound.
13. **Red LED (pin 3):** Blinks (or fades) during alarm mode to provide a visual alert.
14. **Green LED (pin 5):** Remains on during normal operation, indicating normal system status.
15. **Power Supply**
16. The system is powered by a stable 5 V supply from the Arduino's regulator or an external 5 V source.

CALCULATIONS AND EXPLANATIONS

SYSTEM ASSUMPTIONS AND LDR SPECIFICATIONS

Details on specifying high and low light intensities, corresponding LDR resistances and voltages, and explanation of the light sensor security alarm system functioning in relation to the system:

We determined the LDR resistance values for the high light and low light conditions from the key specifications provided in the LDR datasheet excerpt shared here:

The specific details are:

1. Light Resistance (RL)
    - At 10 lux: 20 KΩ to 200 KΩ
    - At 100 lux: 5 00Ω
2. Dark Resistance (RD)
    - 1 MΩ

So for the:

High light condition (100 lux):

- Datasheet directly specifies LDR resistance as 500Ω

Low light condition (10 lux):

- Datasheet specifies a range of 20 KΩ to 200 KΩ
- The conservative higher end value of 200 KΩ for the low light state was taken.

Essentially, the datasheet provides the behavior of LDR resistance at different standard illumination levels of 10 lux and 100 lux which correspond to low light and high light conditions for our security alarm use case.

Calculations Of System

Light Intensity Levels and LDR Parameters

1. High Light Condition (100 lux)
    - LDR Resistance: 500Ω (from datasheet specs)
    - Voltage (with 10k resistor):

$$V_{ldr} = \frac{V_{cc} * R_{ldr}}{R_{ldr} + R} = \frac{5 * 500}{500 + 3700} = 0.595\ mV$$

2. Low Light Condition (10 lux)
    - LDR Resistance: 200kΩ (datasheet specs)

$$V_{ldr} = \frac{V_{cc} * R_{ldr}}{R_{ldr} + R} = \frac{5 * 200\,000}{200\,00 + 3700} = 4.909\ mV$$

Hence, a resistor value of **3700 Ohms** is appropriate to be used in the voltage divider circuit along with the ldr.

Cutoff Frequency Analysis

The LDR has a response time of 50ms in datasheet, which gives a cutoff frequency of:

$$f = \frac{1}{2\pi * \text{Response Time}}$$

$$f = \frac{1}{2\pi * 0.05} = 3.18\ Hz$$

In our analysis, we used the LDR's response time to be 50 ms as stated in the datasheet, corresponding to a cutoff frequency of 3.18 Hz.

PARAMETERS AND SPECIFICATIONS FOR DESIGNING THE LOW PASS BUTTERWORTH (IIR) FILTER FOR REMOVING EXTERNAL NOISE USING THE FDA TOOL IN MATLAB

FDA Tool design specifications:

• Sampling Frequency (Fs): 100 Hz

• Passband Frequency (Fpass): 2 Hz

• Stopband Frequency (Fstop): 5 Hz

• Passband Ripple (Ap): 1 dB (Max allowed passband loss)

• Stopband Attenuation (Ast): 10 dB (Min required stopband attenuation) • Filter order: 2

Filter type:

• 'Lowpass' in design method

• Butterworth.

These parameters helped design a lowpass filter that meets the desired cutoff frequency specification and to remove more noise than even the previous filter.

DIGITAL FILTER DESIGN

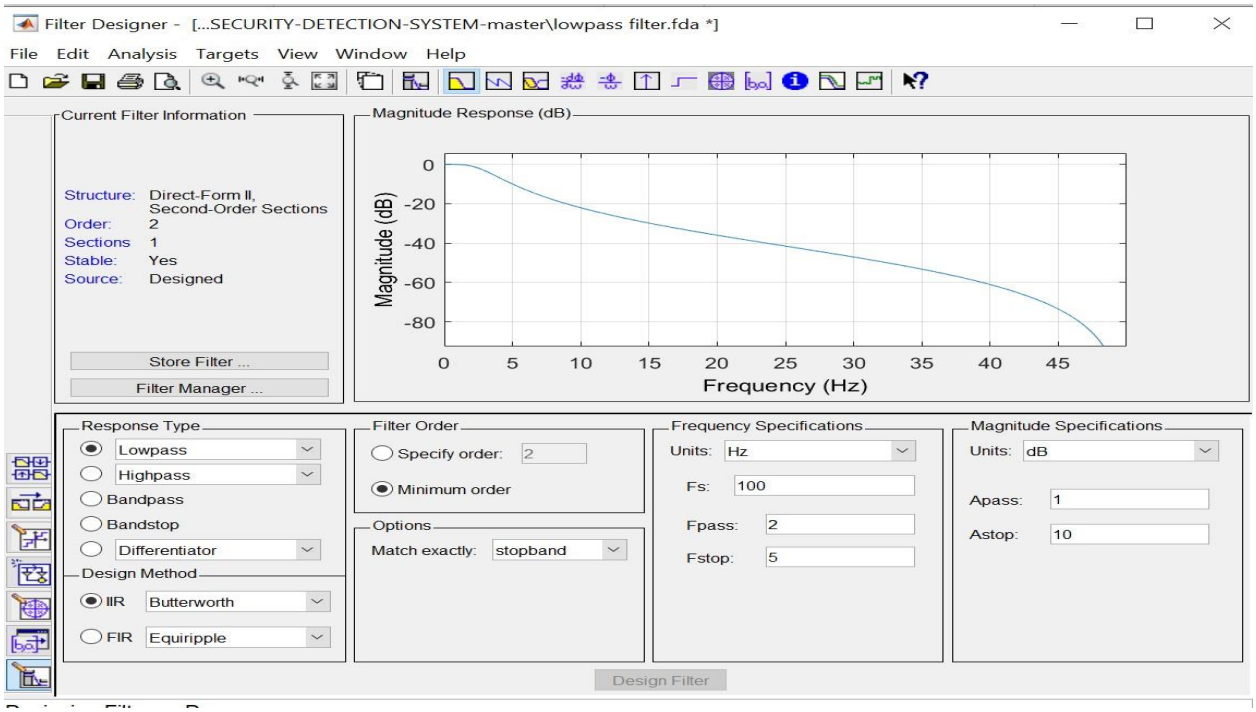


Fig 5: The figure above shows the magnitude response of the filter.

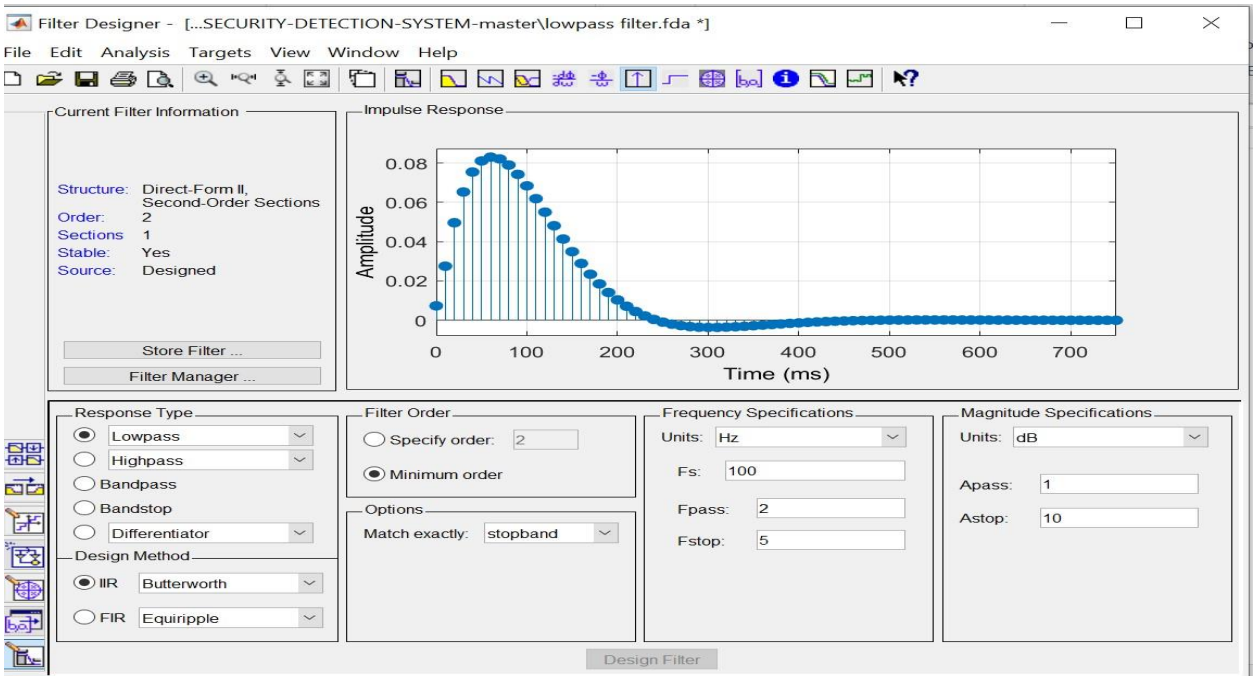


Fig 6: The figure above shows the impulse response of the filter.

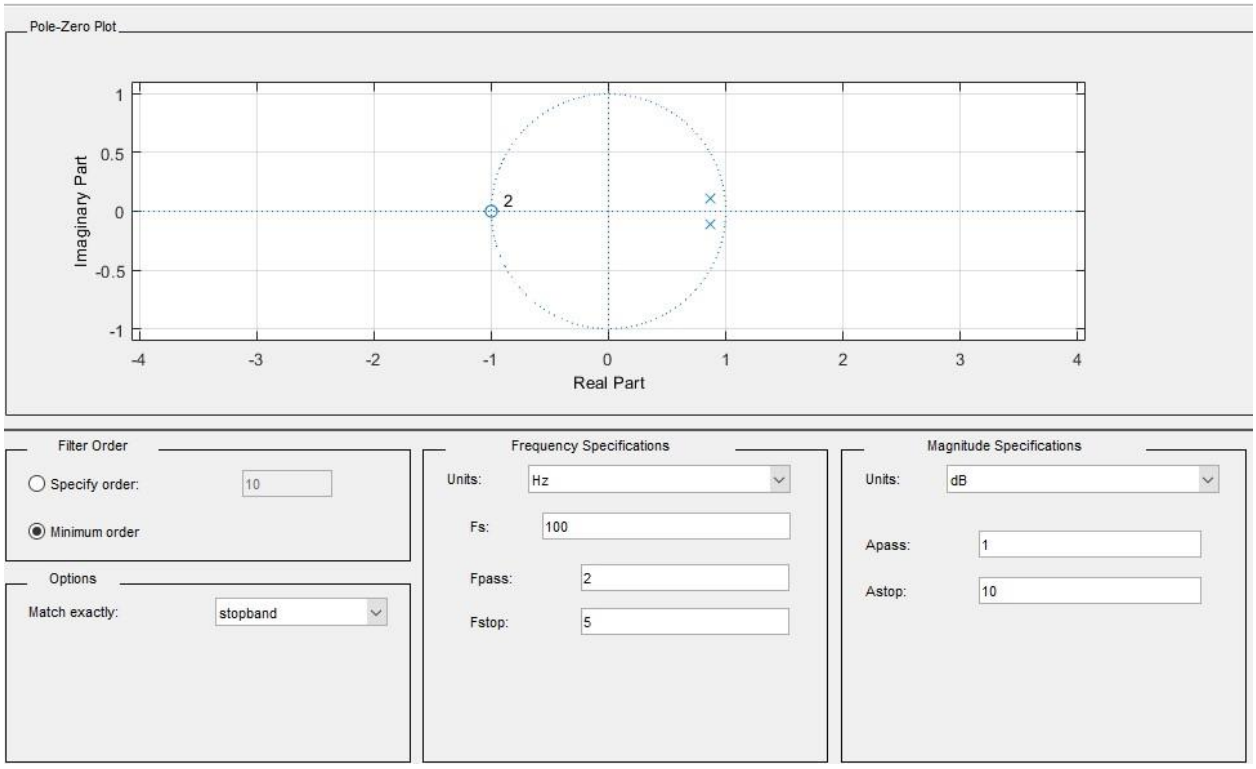


Fig 7: The figure above shows the zero pole map of the filter indicating that the system is stable

```
% Generated by MATLAB(R) 9.10 and Signal Processing Toolbox 8.6.
% Generated on: 22-Aug-2024 21:38:42

% Coefficient Format: Decimal

% Discrete-Time IIR Filter (real)
% -------------------------------
% Filter Structure    : Direct-Form II, Second-Order Sections
% Number of Sections  : 1
% Stable              : Yes
% Linear Phase        : No


SOS Matrix:
1  2  1  1  -1.743260281335717420958530965435784310102  0.772659973978531589011709002079442143 44

Scale Values:
0.007349923160703567166784910824617327307
```

Fig 8: The figure above shows the filter coefficients for the digital filter design

1. Choosing the appropriate components for the 555 Astable timer to achieve the frequency of operation of the LDR which is 3 Hz.

f = 3Hz

$$f = \frac{1.44}{(Ra + 2Rb)C}$$

$$3 = \frac{1.44}{(Ra + 2Rb)C}$$

$$(Ra + 2Rb)C = \frac{1.44}{3}$$

$$(Ra + 2Rb)C = 0.48$$

Choosing a capacitor value of 10 micro-Farad:

$$(Ra + 2Rb) * 10 * 10^{-6} = 0.48$$

$$(Ra + 2Rb) = \frac{1.44}{10 * 10^{-6}}$$

$$(Ra + 2Rb) = 48000$$

Choosing a value of **Ra = 1000 Ω** and **Rb = 23500 Ω:**

$$(1000 + 2(23500)) = 48000$$

Checking:

$$f = \frac{1.44}{(1000 + 2(23500)) * 10 * 10^{-6}} = 3\ Hz$$

2. Timer calculations

1. Nyquist Theorem & Avoiding Aliasing

- The system needs to **detect a 3 Hz flickering pattern** in light intensity from the LDR sensor.
- According to the **Nyquist theorem**, the sampling frequency must be **at least twice the highest frequency** to be detected.

$$f_{sampling} \geq 2\,x\,f_{signal}$$

$$2 * 3 = 6\,Hz$$

  - However, using just **6 Hz** would lead to poor signal reconstruction.
  - A much **higher sampling rate** is preferred for better filtering and noise handling.
- **100 Hz will ensure that at least 33 samples per cycle of the 3 Hz signal** (since **100 / 3 ≈ 33**), which is more than sufficient to analyze signal variations smoothly.

2. Ensuring Smooth Filtering & Detection

- The system uses a **biquad IIR filter** to smooth sensor readings.
- Higher sampling rates allow **better resolution of gradual changes** in light intensity.
- With **100 Hz**, each **3 Hz cycle has multiple well-spaced samples**, ensuring effective filtering and hysteresis-based detection.

3. Real-Time Processing Efficiency

- **100 Hz** balances **speed and computational efficiency**:
  - Too **low (e.g. 10 Hz)** → Poor detection resolution.
  - Too **high (e.g. 1 kHz)** → Wastes CPU cycles, increases power consumption, and may create unnecessary noise.
- The **Arduino can handle 100 Hz easily** while still managing other tasks, including LCD updates, LED controls, and EEPROM logging.

4. Power Consumption & Interrupt Handling

- The **Timer1 interrupt** runs every **10 ms (100 Hz)**.
- This allows the **microcontroller to sleep between readings**, reducing **power usage**.
- A lower frequency (e.g. **10 Hz**) would cause lag in detecting intrusions.
- A higher frequency (e.g. **500 Hz**) would wake up the microcontroller too often, consuming more power.

How the Timer1 Settings will be made to Achieve a frequency of 100 Hz

The **Timer1 ISR (Interrupt Service Routine)** must execute **every 10 ms**. This is done by:

- Selecting **Prescaler = 1024** to slow down the 16 MHz clock.

Using the formula:

$$f_{interrupt} = \frac{f_{CPU}}{Prescaler * (1 + OCR1A)}$$

$$100 = \frac{16\,000\,000}{1024 * (1 + OCR1A)}$$

$$OCR1A = \frac{16\,000\,000}{1024 * 100} - 1 = 156$$

- Setting **OCR1A = 156** in **CTC mode**, will ensure that the timer triggers **exactly every 10 ms**.

Thus, every **10 ms**, an interrupt occurs, allowing the **LDR sensor to be read and processed**.

10.SOFTWARE (ARDUINO CODE) EXPLANATION

Below is the **complete Arduino sketch** used in the system. Each section is described in detail.

```
#include <LiquidCrystal.h>
#include <avr/wdt.h>         // For Watchdog
#include <avr/interrupt.h>   // For Timer1 ISR
#include <avr/sleep.h>       // For Low-Power Sleep
#include <EEPROM.h>          // For Data Logging

/*****************************************************/
/*                 LCD Setup                         */
/*****************************************************/
LiquidCrystal lcd(13, 12, 11, 10, 9, 8);

/*****************************************************/
/*         Biquad Filter Coefficients (SOS)          */
/*         Single-section Direct-Form II             */
/*****************************************************/
static const float b[3] = { 1.0, 2.0, 1.0 };
static const float a[3] = { 1.0, -1.7432602813357174, 0.7726599739785316 };
static const float G = 0.007349923160703567;  // Overall gain

/*****************************************************/
/*             Global States and Variables           */
/*****************************************************/
volatile float lastFilteredValue = 0.0f; // Latest filtered sample
volatile bool sampleReady = false;       // New sample flag

// Direct Form II internal state variables
static float w1 = 0.0f; // w[n-1]
static float w2 = 0.0f; // w[n-2]

// Hysteresis thresholds to avoid erratic switching
float thresholdHigh = 0.50;
float thresholdLow  = 0.45;
```

```
// Detection state variables
bool detection = false;           // Current detection state
bool previousDetection = false;   // For logging transitions

// EEPROM logging variables
int eventCount = 0;               // Count of detection events
// We'll store eventCount at EEPROM address 0 and lastDetectionTime at address
2.
const int eventCountAddress = 0;
const int detectionTimeAddress = 2;  // assuming int is 2 bytes, unsigned long
uses 4 bytes

unsigned long lastDetectionTime = 0; // Time (millis) when last detection
occurred

// Pin definitions
const int sensorPin   = A0; // LDR sensor (analog)
const int buzzerPin   = 6;  // Buzzer (tone output)
const int redLedPin   = 5;  // Red LED (PWM-capable)
const int greenLedPin = 3;  // Green LED (PWM-capable)
const int buttonPin   = 7;  // Button to retrieve detection info

// Creative alarm mode variables
bool alarmActive = false;
unsigned long alarmStartTime = 0;
const unsigned long alarmDuration = 5000; // Alarm mode lasts 5 seconds

/*************************************************/
/*        Single-Sample Direct Form II Filter        */
/*************************************************/
float directForm2SingleSample(float x)
{
  float input = G * x;
  float a1 = a[1];
  float a2 = a[2];

  // Compute filter output
  float w0 = input - a1 * w1 - a2 * w2;
  float y0 = b[0] * w0 + b[1] * w1 + b[2] * w2;

  // Shift state variables
  w2 = w1;
  w1 = w0;

  return y0;
}
```

```
/*****************************************************/
/*                  Timer1 ISR                       */
/*****************************************************/
ISR(TIMER1_COMPA_vect)
{
  // Read sensor (0-1023) and normalize to 0.0-1.0
  float x_in = analogRead(sensorPin) / 1023.0;

  // Filter new sample
  float y_out = directForm2SingleSample(x_in);
  lastFilteredValue = y_out;

  // Signal that a new sample is ready
  sampleReady = true;
}

/*****************************************************/
/*                Setup Timer1                       */
/*****************************************************/
void setupTimer1()
{
  cli(); // Disable global interrupts

  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1  = 0; // Reset counter

  // For a 16MHz clock with prescaler 1024:
  // 16,000,000 / 1024 ≈ 15625 Hz.
  // For ~100Hz interrupts: OCR1A = 156.
  OCR1A = 156;

  TCCR1B |= (1 << WGM12);            // CTC mode
  TCCR1B |= (1 << CS12) | (1 << CS10); // Prescaler 1024
  TIMSK1 |= (1 << OCIE1A);            // Enable Timer1 compare interrupt

  sei(); // Re-enable interrupts
}

/*****************************************************/
/*             Setup Watchdog Timer                  */
/*****************************************************/
void setupWatchdog()
{
  wdt_enable(WDTO_2S); // 2-second watchdog timeout
}

/*****************************************************/
```

```
/*                    Low Power                          */
/*******************************************************/
void goToSleep()
{
  set_sleep_mode(SLEEP_MODE_IDLE);
  sleep_enable();
  sleep_cpu();
  sleep_disable();  // Resume after wake-up
}

/*******************************************************/
/*                 EEPROM Logging                      */
/*******************************************************/
void loadDetectionInfoFromEEPROM()
{
  EEPROM.get(eventCountAddress, eventCount);
  EEPROM.get(detectionTimeAddress, lastDetectionTime);
}

void saveDetectionInfoToEEPROM()
{
  EEPROM.put(eventCountAddress, eventCount);
  EEPROM.put(detectionTimeAddress, lastDetectionTime);
}

/*******************************************************/
/*           Print Detection Info on LCD               */
/*******************************************************/
void printDetectionInfo() {
  // Retrieve detection info from EEPROM
  int count;
  unsigned long time;
  EEPROM.get(eventCountAddress, count);
  EEPROM.get(detectionTimeAddress, time);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Detections: ");
  lcd.print(count);

  lcd.setCursor(0, 1);
  lcd.print("Time: ");
  // Convert millis to seconds for display:
  lcd.print(time / 1000);
  lcd.print(" sec");

  Serial.print("Retrieved - Count: ");
  Serial.print(count);
```

```
  Serial.print(", Last Detection Time: ");
  Serial.print(time / 1000);
  Serial.println(" sec");

  // Wait a moment to allow the user to read the info
  delay(3000);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Light Intensity");
}

/*****************************************************/
/*                       Setup                       */
/*****************************************************/
void setup()
{
  Serial.begin(9600);

  // Configure pin modes
  pinMode(sensorPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  pinMode(redLedPin, OUTPUT);
  pinMode(greenLedPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);  // Button with internal pull-up

  // Initialize LCD
  lcd.begin(16, 2);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Light Intensity");

  // Setup Timer1 and Watchdog Timer
  setupTimer1();
  setupWatchdog();

  // Load previous detection info from EEPROM
  loadDetectionInfoFromEEPROM();
  Serial.println("System Initialized");
  Serial.print("Loaded eventCount = ");
  Serial.print(eventCount);
  Serial.print(", Last Detection Time = ");
  Serial.print(lastDetectionTime / 1000);
  Serial.println(" sec");
}

/*****************************************************/
/*                       Loop                        */
/*****************************************************/
```

```
void loop()
{
  wdt_reset(); // Reset watchdog timer

  // Check for button press to display detection info
  if (digitalRead(buttonPin) == LOW) {
    // Simple debounce delay
    delay(50);
    if (digitalRead(buttonPin) == LOW) {
      printDetectionInfo();
    }
  }

  // Process new sensor sample and update detection state.
  if (sampleReady)
  {
    sampleReady = false;

    // Hysteresis: update detection state based on filtered value.
    if (!detection && lastFilteredValue > thresholdHigh)
      detection = true;
    else if (detection && lastFilteredValue < thresholdLow)
      detection = false;

    // Trigger alarm mode on rising edge.
    if (detection && !alarmActive)
    {
      alarmActive = true;
      alarmStartTime = millis();
      eventCount++;
      // Record the time (millis) of this detection event
      lastDetectionTime = millis();
      saveDetectionInfoToEEPROM();
      Serial.print("Detection event #");
      Serial.println(eventCount);
    }
  }

  // If alarm mode is active, run creative outputs.
  if (alarmActive)
  {
    unsigned long t = millis() - alarmStartTime;

    // Buzzer modulation (1-second period)
    const unsigned long modPeriodBuzzer = 1000;
    float angleBuzzer = 2.0 * PI * (float)(t % modPeriodBuzzer) /
(float)modPeriodBuzzer;
    unsigned int freq = 400 + (unsigned int)(200 * sin(angleBuzzer));
```

```
    tone(buzzerPin, freq);

    // Red LED fading using PWM: Fade in and out over a 2-second cycle
    const unsigned long fadePeriod = 2000;
    unsigned long fadeTime = t % fadePeriod;
    int redPWM;
    if (fadeTime < fadePeriod / 2)
      redPWM = map(fadeTime, 0, fadePeriod / 2, 0, 255);
    else
      redPWM = map(fadeTime, fadePeriod / 2, fadePeriod, 255, 0);
    analogWrite(redLedPin, redPWM);

    // Turn off the green LED during alarm mode
    analogWrite(greenLedPin, 0);

    // Update LCD with alarm status.
    lcd.setCursor(0, 1);
    lcd.print("ALARM: DETECTED    ");

    // Print alarm info to Serial.
    Serial.print("ALARM Mode: Filtered Value = ");
    Serial.print(lastFilteredValue, 3);
    Serial.print(" (Freq: ");
    Serial.print(freq);
    Serial.println(")");

    // End alarm mode after the specified duration.
    if (t >= alarmDuration)
    {
      alarmActive = false;
      noTone(buzzerPin);
      analogWrite(redLedPin, 0);
    }
  }
  else // Normal mode.
  {
    noTone(buzzerPin);
    analogWrite(redLedPin, 0);
    // For the green LED, set it to constant full brightness.
    analogWrite(greenLedPin, 255);

    lcd.setCursor(0, 1);
    lcd.print("Status: Normal     ");

    Serial.print("Filtered Value = ");
    Serial.print(lastFilteredValue, 3);
    Serial.print(" => Detection = ");
    Serial.println(detection);
```

```
  }

  goToSleep();
}
```

CODE BREAKDOWN

1. Libraries

- **LiquidCrystal.h:** For interfacing with the 16×2 LCD.
- **avr/wdt.h, avr/interrupt.h, avr/sleep.h:** Provide watchdog timer, interrupt handling, and low-power sleep functionalities.
- **EEPROM.h:** Used for non-volatile data storage (detection count and last detection time).

2. Biquad Filter Coefficients

- **Arrays b[3] and a[3] with gain G:**
  - These define a second-order IIR filter in Direct Form II used to smooth the LDR sensor readings.
  - The filter reduces noise, ensuring that only a proper signal (e.g., a 3 Hz pattern) triggers an alarm.

3. Global Variables

- **lastFilteredValue & sampleReady:**
  - `lastFilteredValue` holds the filtered sensor value.
  - `sampleReady` is set by the Timer1 ISR when a new sample is available.
- **Direct Form II Internal Variables (w1, w2):**
  - Used for maintaining the filter's state.
- **Hysteresis Thresholds (thresholdHigh, thresholdLow):**
  - Define the upper and lower bounds to decide when an intruder is detected.
- **Detection State Variables (detection, previousDetection):**
  - `detection` indicates whether the filtered signal crosses the threshold.
- **EEPROM Logging Variables:**
  - `eventCount`: The number of detection events, stored at EEPROM address 0.
  - `lastDetectionTime`: The time (in milliseconds) when the last detection occurred, stored at EEPROM address 2.
- **Pin Definitions:**
  - `sensorPin` (A0): Reads the analog LDR signal.
  - `buzzerPin` (6): Drives the buzzer for the alarm tone.
  - `redLedPin` (5): Controls the red LED (PWM used for fading effect).
  - `greenLedPin` (3): Controls the green LED (constant full brightness in normal mode).
  - **New:** `buttonPin` (7): Connected to a normally open push-button (with internal pull-up enabled) used to trigger display of the detection count and last detection time.
- **Creative Alarm Mode Variables:**

- `alarmActive` and `alarmStartTime` manage the alarm state and duration (set to 5000 ms).

4. Timer1 ISR

- **Purpose:**
  - Fires at approximately 100 Hz (every 10 ms) to sample the sensor.
  - Reads the analog LDR value, applies the biquad filter, updates `lastFilteredValue`, and sets `sampleReady = true`.
- **Relevance:**
  - Decouples the sensor sampling from the main loop, ensuring consistent real-time performance.

5. setupTimer1()

- **Configuration:**
  - Configures Timer1 in CTC mode with a prescaler of 1024 and sets `OCR1A = 156` to achieve ~100 Hz.
- **Result:**
  - The Timer1 ISR is triggered every 10 ms for precise sensor sampling.

6. setupWatchdog()

- **Function:**
  - Enables the watchdog timer with a 2-second timeout.
- **Purpose:**
  - Ensures that if the main loop hangs, the MCU resets, maintaining system reliability.

7. goToSleep()

- **Function:**
  - Puts the MCU into IDLE sleep mode between Timer1 interrupts.
- **Purpose:**
  - Reduces power consumption by allowing the CPU to sleep when no active processing is required.

8. EEPROM Logging Functions

- **loadDetectionInfoFromEEPROM() & saveDetectionInfoToEEPROM():**
  - These functions are used to retrieve and store the detection count and the last detection time into non-volatile EEPROM memory.
- **printDetectionInfo():**
  - When the push-button is pressed, this function reads the stored detection count and last detection time from EEPROM, then displays them on the LCD (and prints them to Serial).
  - The displayed time is converted from milliseconds to seconds for readability.

9. Main Loop

- **Button Handling:**
  - The loop checks if the push-button (on pin 7) is pressed (using the internal pull-up).
  - When pressed (after a simple debounce delay), the `printDetectionInfo()` function is called to display the stored detection count and last detection time.
- **Sensor Processing & Hysteresis:**
  - If a new sensor sample is available (`sampleReady` is true), the code clears the flag and processes the reading using the biquad filter.
  - Hysteresis logic compares `lastFilteredValue` to `thresholdHigh` and `thresholdLow` to update the `detection` flag.
- **Alarm Mode Trigger:**
  - On the rising edge of detection (i.e., detection changes from false to true), alarm mode is triggered:
    - `alarmActive` is set to true, the alarm start time is recorded, `eventCount` is incremented, and `lastDetectionTime` is updated to `millis()`.
    - These values are then saved into EEPROM.
- **Alarm Mode Operation:**
  - If `alarmActive` is true, the code performs the following:
    - **Buzzer:** Modulates the tone with a 1-second cycle (using a sine function) to produce a varying frequency between 400 and 600 Hz.
    - **Red LED:** Uses a PWM fade loop (over a 2-second cycle) to fade in and out, creating a visible alarm effect.
    - **Green LED:** Turned off during alarm mode.
    - **LCD:** Displays "ALARM: DETECTED".
    - **Duration Check:** After 5 seconds (`alarmDuration`), the alarm mode ends (turning off the buzzer and red LED).
- **Normal Mode Operation:**
  - When no alarm is active, the system maintains normal outputs:
    - **Buzzer:** Off.
    - **Red LED:** Off.
    - **Green LED:** Kept at constant full brightness (using PWM set to 255).
    - **LCD:** Displays "Status: Normal".
- **Low-Power Sleep:**
  - At the end of each loop cycle, `goToSleep()` is called, putting the MCU in IDLE mode until the next Timer1 interrupt.

SYSTEM OPERATION SUMMARY

- **At Startup:**
  - The system initializes all peripherals (LCD, Timer1, Watchdog, EEPROM, and pin modes) and loads any previously stored detection data from EEPROM.
- **During Operation:**
  - The LDR sensor is sampled every 10 ms, and its value is filtered to reduce noise.
  - Hysteresis logic determines whether an intrusion is detected, triggering alarm mode on a rising edge.

- **Alarm Mode:**
  - For 5 seconds, the buzzer produces a modulated tone, the red LED fades in and out via PWM, and the LCD displays an alarm message.
  - The detection event (count and time) is stored in EEPROM.
- **Normal Mode:**
  - The green LED is kept on (steady) to indicate that the system is operating normally.
- **User Interaction:**
  - When the button on pin 7 is pressed, the stored detection count and last detection time are retrieved from EEPROM and displayed on the LCD.
- **Power Management:**
  - The watchdog timer ensures the system resets if it hangs, and the CPU enters a low-power idle state between Timer1 interrupts.

DETAILED EXPLANATIONS

1. Timer (Timer1)

Where Used

- The system uses **Timer1** on the Arduino (an AVR microcontroller) in **CTC (Clear Timer on Compare) mode**.

Why Used

- To **sample the LDR sensor** at a consistent rate (~100 Hz).
- Ensures **real-time** and **periodic** reading of the sensor, independent of the main loop.

How It Operates

1. **Configuration**:
   - OCR1A = 156 with a prescaler of 1024 gives a 10 ms interval (~100 Hz).
   - TCCR1B |= (1 << WGM12); sets CTC mode.
   - TIMSK1 |= (1 << OCIE1A); enables the compare match interrupt.
2. **Interrupt Trigger**:
   - Every time Timer1 counts to 156, it **fires an interrupt**.
3. **Result**:
   - The **ISR (TIMER1_COMPA_vect)** reads the LDR, applies the filter, and sets sampleReady = true.
   - This decouples **sensor sampling** from the main loop timing, ensuring **precise intervals**.

2. PWM (Pulse Width Modulation)

Where Used

- The **Arduino pins** 3 and 5 are labeled as "PWM-capable."
- In the provided code, the **buzzer** uses the tone() function (not raw PWM), and the **red LED** is blinking digitally (on/off).

Red LED (Alarm Mode):

- We use a 3-second cycle for fading.
- The code calculates fadeTime as the remainder of the elapsed alarm time modulo 2000 ms.
- If fadeTime is in the first half of the period, the PWM value ramps from 0 to 255; if in the second half, it ramps back down from 255 to 0.
- This is done with the map() function, and the resulting value is sent to the red LED via analogWrite(), creating a smooth fade visible to the human eye.

Green LED (Normal Mode):

- In normal mode, the green LED is set to full brightness with analogWrite(greenLedPin, 255).
- The inherent high-frequency PWM (typically 490 Hz or 980 Hz) ensures that the flickering is not visible.

3. Interrupt

Where Used

- The **Timer1 compare match** interrupt is the primary interrupt in use.

Why Used

- Interrupts let the system **respond immediately** when the timer reaches a compare value, without polling in the main loop.
- This ensures the **sensor reading** and **filtering** happen **on a fixed schedule** (~100 Hz).

How It Operates

1. **Timer1** is configured in CTC mode.
2. **When the timer matches OCR1A**, the **ISR** TIMER1_COMPA_vect executes.
3. **Inside the ISR**, the code reads the analog sensor, filters it, and sets sampleReady = true.
4. **Back in the main loop**, we check sampleReady to process new data.

4. Watchdog

Where Used

- The system calls wdt_enable(WDTO_2S); in setupWatchdog() to enable a **2 s watchdog**.

Why Used

- To **prevent the system from hanging** if the main loop ever locks up or if interrupts fail.

- This is important in **security** or **mission-critical** applications where a freeze would be unacceptable.

HOW IT OPERATES

1. The watchdog timer counts down from 2 seconds.
2. **In the main loop**, the code calls wdt_reset() regularly.
3. If the loop or interrupts **stop running** and wdt_reset() is not called in time, the watchdog **expires** and **resets** the microcontroller.
4. The system restarts, ensuring it never remains in a locked or unresponsive state.

SUMMARY

- **Timer1** (in CTC mode) **interrupts** every 10 ms, providing **precise sensor sampling**.
- **PWM** is available on pins 3 and 5 for **LED/buzzer** control (though the code currently uses tone() for the buzzer and digital blink for the LED).
- The **interrupt** ensures **real-time** operation and keeps the main loop **non-blocking**.
- The **watchdog** maintains **system reliability** by resetting the MCU if the code hangs, ensuring the security system remains operational at all times.

11.CONCLUSION

The system's light detection mechanism relies on an LDR and a comparator stage to ensure that the Arduino receives a stable signal corresponding to the ambient light intensity. By adjusting resistor values, including R1, R2, and the variable resistor RV2, the threshold range can be calibrated to suit specific environmental conditions. This calibration allows the system to distinguish between normal fluctuations in light and significant changes that may indicate a security breach.

To enhance detection accuracy, a biquad filter is implemented to reduce noise, ensuring that the signal remains stable and reliable. While this filtering stage is beneficial in preventing false alarms, it is possible to simplify the design by omitting the filter, though this may increase the likelihood of erroneous detections caused by minor fluctuations in light intensity.

The system employs hysteresis to prevent rapid on/off switching, which can occur due to small variations in detected light levels. By defining two thresholds—threshold High and threshold Low—the system ensures that once an alarm is triggered, it remains active until the light intensity drops below a lower predefined level. This approach minimizes erratic behavior and provides a more robust response to actual security threats. Fine-tuning these thresholds allows the system to adapt to different lighting environments and levels of sensitivity.

When an intrusion is detected, the system enters alarm mode, activating both a buzzer and a red LED to provide audible and visual alerts. The buzzer produces a modulated tone, while the LED blinks to indicate a security breach. The duration of the alarm is set to five seconds by default, but this can be adjusted through the alarm duration parameter to extend or shorten the response time based on user preferences.

To optimize efficiency and ensure reliable operation, the system incorporates low-power functionality and a watchdog timer. During periods of inactivity, the microcontroller is put into sleep mode to reduce power consumption, making the system suitable for battery-powered applications. Additionally, the watchdog timer enhances reliability by automatically resetting the microcontroller in the event of a software malfunction, preventing system failures and ensuring continuous operation.

For **Proteus Simulation**:

- Ensure the **Arduino Uno** is clocked at **16 MHz**.
- Double-check that the **op-amp** or comparator has enough supply voltage to output near **5 V** when the LDR is "high."
- The buzzer in Proteus might be a "virtual" device; check the output on a simulated oscilloscope if needed.

12.REFERENCES

1. Putri, M., & Aryza, S. (2018). Design of security tools using sensor light dependent resistor (LDR) through mobile phone. International Journal for Innovative Research in Multidisciplinary Field, 4(10), 1681.

2. Dixit, J., Katiyar, D., & Goel, G. (2021). Automatic street light controller system using LDR and PIR sensor. International Journal of Creative Research Thoughts, 9(4), 9492.

3. Misra, D. S. (2021). Research on automatic light control system using LDR. International Journal of Creative Research Thoughts, 9(4), 1323.

4. Hossain, A. (2021). Automatic light detector using LDR in Proteus. The Engineering Projects.

5. Utsource Parts. (2020). Laser based security system with Arduino. Maker Pro.

6. Electronics Tutorials. (n.d.). Light sensor including photocell and LDR sensors.